Naad Music Player

Swar Sawalkar
Vidyalankar Institute of Technology of Affiliation
Mumbai University
Wadala, Mumbai
sawalkarswar@gmail.com

***Abstract*—The Naad Music Player is a desktop-based application developed using Java Swing that provides an efficient and visually appealing platform for playing and managing audio files. The system integrates a modern graphical user interface with robust backend functionality to ensure smooth music playback and user interaction. The application supports multiple audio formats, playlist management, shuffle and repeat modes, and equalizer presets. This project demonstrates the practical implementation of object-oriented programming, event-driven architecture, and multithreading in Java. The proposed system aims to enhance user experience while maintaining simplicity and performance.**

Keywords—Java Swing, Music Player, Audio Playback, GUI, Multimedia Application, Object-Oriented Programming

I. INTRODUCTION

A. *With the increasing demand for digital media consumption, music players have become an essential application in modern computing systems. The Naad Music Player is designed to provide a simple yet powerful solution for managing and playing audio files on a desktop environment. Unlike complex commercial applications, this project focuses on implementing core functionalities using fundamental Java concepts. The system combines aesthetic user interface design with efficient backend processing, making it suitable for both learning and practical use.*

1) With the rapid growth of digital media, music players have become an essential part of everyday computing. Users require applications that are not only functional but also visually appealing and easy to use. While commercial music players provide advanced features, they are often complex and resource-intensive.

2) The Naad Music Player is designed as a lightweight yet feature-rich application that focuses on usability, performance, and modern design. The project demonstrates how a desktop application can be built using core Java technologies such as Swing and AWT, without relying on external frameworks. It also highlights important programming concepts such as event-driven design, multithreading, and modular architecture.

Literature Review

Several music player applications exist in the market, including VLC Media Player, Windows Media Player, and Spotify. These applications offer advanced features such as streaming, equalization, and cloud integration. However, they are often complex and rely on large external libraries.

It's recommended to insert figures inside a text box. So your figure would go here (adjusting the text box to the appropriate size to match).

Research in multimedia applications emphasizes efficient audio processing, responsive user interfaces, and modular system design. Java-based music players have been widely studied for their portability and simplicity. The Java Sound API provides a foundation for handling audio playback, while Swing enables GUI development.
This project builds upon these concepts by creating a simplified yet effective music player that demonstrates core principles without unnecessary complexity.

Problem Statement

Many existing music players are either too complex for beginners or lack customization options. There is a need for a simple, lightweight, and customizable music player that demonstrates core multimedia concepts while providing a modern user experience.

Objectives

The objectives of this project are:

- *To develop a desktop-based music player using Java*
- *To design an attractive and modern graphical user interface*
- *To implement efficient music playback using Java Sound API*
- *To manage music libraries and playlists effectively*
- *To incorporate features such as shuffle, repeat, and equalizer presets*
- *To ensure smooth performance using multithreading*

System Architecture

*The system follows a **modular architecture** consisting of two main components:*

II. 1. Backend (MusicPlayer.java)

The backend is responsible for:

- *Audio playback control*
- *Song library management*
- *Playlist handling*
- *Shuffle and repeat logic*
- *Volume control*

III. 2. Frontend (BeautifulMusicPlayer.java)

The frontend is responsible for:

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- *Displaying the graphical interface*
- *Handling user interactions*
- *Rendering album art and controls*
- *Updating UI dynamically*

IV. Architecture Flow

User Action → UI (Swing) → Backend Logic → Audio Playback → UI Update

Methodology

The development of the Naad Music Player follows an object-oriented and event-driven approach:

1. ***File Handling***
   Audio files are loaded from local directories and stored in data structures.
2. ***Audio Playback***
   Java Sound API is used to play audio using Clip and AudioInputStream.
3. ***Multithreading***
   Playback runs in a separate thread to prevent UI blocking.
4. ***Event Handling***
   User actions such as button clicks are handled using event listeners.
5. ***UI Design***
   Swing and AWT are used to create custom components such as:
   - *Gradient backgrounds*
   - *Circular album art*
   - *Glassmorphism panels*

Implementational Details

V. 1. Classes Used

- *MusicPlayer → Core logic*
- *Song → Stores metadata*
- *Playlist → Manages song groups*
- *BeautifulMusicPlayer → UI*

VI. 2. Data Structures

- *ArrayList → Store songs*
- *HashMap → Manage playlists*
- *TreeSet → Store unique artists*

VII. 3. Algorithms

- *Shuffle → Random index selection*
- *Repeat → Conditional playback logic*
- *Search → String matching*

Features

- ***1. Playback Control***
- *Play, Pause, Stop*
- *Next / Previous*
- *Seek functionality*
- ***2. Playlist Management***
- *Create playlists*
- *Add/remove songs*
- *Recently played tracking*
- ***3. Smart Features***
- *Shuffle mode*
- *Repeat modes (Off, One, All)*
- ***4. Equalizer***
- *Preset-based audio modes*
- ***5. UI Features***
- *Gradient background*
- *Circular album art*
- *Glassmorphism playlist*
- *Animated buttons*

Results

The application successfully performs all core functionalities. Audio playback is smooth, and the UI remains responsive due to multithreading. The modular design ensures easy maintenance and scalability.
The UI enhances user experience by providing a modern and clean design. However, limitations such as basic metadata extraction and lack of streaming features indicate areas for improvement.

Advantages

- *Lightweight and efficient*
- *Cross-platform compatibility*
- *No external dependencies*
- *Clean and modular design*
- *Easy to use interface*

Limitations

- *Limited metadata support (no ID3 tags)*
- *No online streaming*
- *Basic equalizer functionality*
- *Limited audio format optimization*

Future Scope

Future improvements can include:

- *Integration with music streaming services*
- *Advanced metadata extraction*
- *Playlist saving and loading*
- *Real-time equalizer adjustments*
- *Mobile application version*
- *Cloud synchronization*

Conclusion

The Naad Music Player demonstrates the successful development of a multimedia desktop application using core Java technologies. It integrates backend logic with a modern user interface to provide a complete and functional system. The project highlights important concepts such as object-oriented programming, multithreading, and event-driven design, making it a valuable learning experience and a strong foundation for future development.

Referrences

- *Oracle, Java Documentation*
- *Java Swing Tutorials, Oracle*
- *Java Sound API Guide*

Table Head	Table Column Head		
	Table column subhead	***Subhead***	***Subhead***
text	Text[a]		

a) And this is a level 4 heading: It's recommended to write your text in a separate document and then add it to this template once it's complete. When copying text into the template from another document, make sure that the

Fig. 1. This is a figure caption. It appears directly underneath the figure.

appropriate style is still applied to each section, reapplying styles if necessary.

b) This is another level 4 heading: It's also possible to add bullet points when appropriate, using the "bullet list" style:

- Treat the word "data" as plural, not singular.
- For example, "the data indicate that …"

B. *This Is Another Level 2 Heading*

Place any figures or tables you use at the top or bottom of a column. Don't place them in the middle of a column. If particularly wide, a table or figure can span across both columns. Insert a table or figure after the point where it is first cited in the text.

When inserting a figure, such as a photograph or infographic, use 8 pt. Times New Roman for any labeling text within the image and for the figure caption. You can see an example of a figure caption in Fig. 1, above. Refer to figures like that, using the abbreviation "Fig." and the figure's number.

A table heading (using the "table head" style) appears above a table. This will automatically number the table for you. Any footnotes appear below the table, using the "table footnote" style. Footnotes are indicated by superscript lowercase letters within the table. An example of a table can be seen in Table I, below.

VIII. This Is Another Level 1 Heading

All the headings in the main body of your paper are numbered (automatically).

Another type of heading is the "component heading", which is used for other components that aren't part of the main text. These are usually your acknowledgments and your references, which you can see examples of below. These headings are not numbered. The correct styling for them can be applied using the "Heading 5" style, which is the same as the "Heading 1" style but without numbering.

TABLE I. THIS IS THE HEADING FOR A TABLE

[a] This is a table footnote.

You can cite your references in text by including the corresponding number, in square brackets [1]. If you need to cite a specific part of the source, you can include a page number [2, p. 13] or range [3, pp. 41–56].

Acknowledgments

"Acknowledgment(s)" is spelled without an "e" after the "g" in American English.

As you can see, the formatting ensures that the text ends in two equal-sized columns rather than only displaying one column on the last page.

This template was adapted from those provided by the IEEE on their own website.

References

[1] D. V. Lindberg and H. K. H. Lee, "Optimization under constraints by applying an asymmetric entropy measure," *J. Comput. Graph. Statist.*, vol. 24, no. 2, pp. 379–393, Jun. 2015, doi: 10.1080/10618600.2014.901225.

[2] B. Rieder, *Engines of Order: A Mechanology of Algorithmic Techniques*. Amsterdam, Netherlands: Amsterdam Univ. Press, 2020.

[3] I. Boglaev, "A numerical method for solving nonlinear integro-differential equations of Fredholm type," *J. Comput. Math.*, vol. 34, no. 3, pp. 262–284, May 2016, doi: 10.4208/jcm.1512-m2015-0241.

Make sure to remove all placeholder and explanatory text from the template when you add your own text. This text should not be here in the final version!